

PROJECT REPORT

4-to-2 Encoder & 2-to-4 Decoder Design in Verilog HDL

Combinational Logic Design, Functional Simulation & Verification

Author: Ayush Sikarwar

B.Tech, Electronics and Communication Engineering

LinkedIn: [linkedin.com/in/ayush-sikarwar-a89878413](https://www.linkedin.com/in/ayush-sikarwar-a89878413)

Repository: github.com/ayushsikarwar3/Encoder-Decoder-Design

License: MIT License

Report generated: July 11, 2026

1. Introduction

This report documents the design, simulation, and functional verification of two fundamental combinational logic circuits implemented in Verilog HDL: a 4-to-2 Encoder and a 2-to-4 Decoder. The project, authored by Ayush Sikarwar (B.Tech, Electronics and Communication Engineering) and published on GitHub, serves as a practical foundation for topics in Digital Electronics, FPGA development, and VLSI design.

The repository is publicly hosted at github.com/ayushsikarwar3/Encoder-Decoder-Design and is released under the MIT License. It contains two Verilog design modules, a combined testbench, a project README with theory and truth tables, and a simulation-output screenshot.

2. Project Objective

- Design a 4-to-2 Encoder using Verilog HDL.
- Design a 2-to-4 Decoder using Verilog HDL.
- Simulate both circuits using an HDL simulator such as Xilinx Vivado or ModelSim.
- Verify circuit functionality using a dedicated testbench.
- Build practical understanding of implementing combinational logic in a hardware description language.

3. Theoretical Background

3.1 Encoder

An encoder is a combinational logic circuit that converts 2^n input lines into n output lines, generating a binary code that corresponds to the single active input line. The 4-to-2 encoder in this project accepts four input lines and produces two output lines; for correct operation only one input is expected to be HIGH at any given time.

3.2 Decoder

A decoder performs the reverse operation of an encoder: it accepts a binary code as input and activates exactly one of several output lines corresponding to that code. The 2-to-4 decoder in this project accepts a 2-bit binary input and drives one of four output lines HIGH.

4. Design Specification and Truth Tables

4.1 4-to-2 Encoder Truth Table

Inputs: D3, D2, D1, D0 | Outputs: Y1, Y0

D3	D2	D1	D0	Y1	Y0
----	----	----	----	----	----

0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

4.2 2-to-4 Decoder Truth Table

Inputs: A1, A0 | Outputs: Y3, Y2, Y1, Y0

A1	A0	Y3	Y2	Y1	Y0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

5. Verilog Implementation

5.1 4-to-2 Encoder Module (“4-to-2 encoder.v”)

The encoder is implemented as a combinational always block using a case statement that maps each valid one-hot input pattern to its corresponding 2-bit binary output. A default case is included to handle undefined / invalid input combinations.

```
module encoder_4to2(
    input [3:0] d,
    output reg [1:0] y
);
always @(*) begin
    case(d)
        4'b0001: y = 2'b00;
        4'b0010: y = 2'b01;
        4'b0100: y = 2'b10;
        4'b1000: y = 2'b11;
        default: y = 2'b00;
    endcase
end
endmodule
```

5.2 2-to-4 Decoder Module (“2 -to-4 decoder.v”)

The decoder is implemented as a combinational always block using a case statement that activates one of four one-hot output bits based on the 2-bit binary input address.

```
module decoder_2to4(
    input [1:0] a,
    output reg [3:0] y
);
always @(*) begin
    case(a)
        2'b00: y = 4'b0001;
        2'b01: y = 4'b0010;
        2'b10: y = 4'b0100;
        2'b11: y = 4'b1000;
    endcase
end
endmodule
```

6. Testbench (“testbench.v”)

A single unified testbench instantiates both the encoder and decoder modules and exercises them with all valid input combinations, applying each stimulus for 10 time units before moving to the next.

```

module tb_encoder_decoder;

reg [3:0] d;
wire [1:0] y_enc;

reg [1:0] a;
wire [3:0] y_dec;

encoder_4to2 ENC(
    .d(d),
    .y(y_enc)
);

decoder_2to4 DEC(
    .a(a),
    .y(y_dec)
);

initial begin

// Encoder Testing
d = 4'b0001; #10;
d = 4'b0010; #10;
d = 4'b0100; #10;
d = 4'b1000; #10;

// Decoder Testing
a = 2'b00; #10;
a = 2'b01; #10;
a = 2'b10; #10;
a = 2'b11; #10;

$finish;

end

endmodule

```

7. Simulation Results

The testbench applies every valid input combination to both modules. The expected simulation outcomes, matching the truth tables in Section 4, are summarized below.

7.1 Encoder Simulation

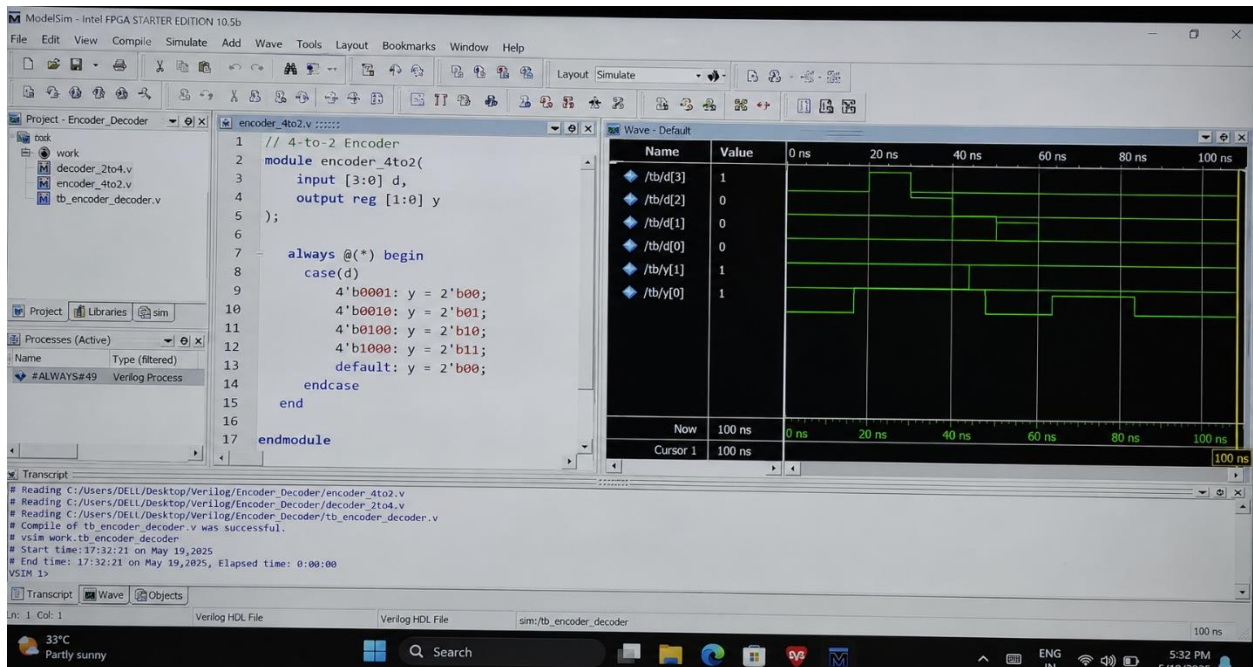
Input (D3 D2 D1 D0)	Output (Y1 Y0)
0001	00
0010	01
0100	10
1000	11

7.2 Decoder Simulation

Input (A1 A0)	Output (Y3 Y2 Y1 Y0)
00	0001
01	0010
10	0100
11	1000

7.3 Simulation Waveform / Screenshot

The repository includes a simulation screenshot (screenshot.png.png) captured from the HDL simulator, showing the waveform / output console confirming that both modules produced results consistent with the truth tables above



8. Repository Structure

File	Description
4-to-2 encoder.v	Verilog source for the 4-to-2 encoder module

2-to-4 decoder.v	Verilog source for the 2-to-4 decoder module
testbench.v	Unified testbench instantiating and stimulating both modules
screenshot.png.png	Simulation output / waveform screenshot
README.md	Project documentation: overview, theory, truth tables, code
LICENSE	MIT License text

Repository composition: 100% Verilog. Commit history: 26 commits on the main branch. No releases have been published, and there are currently no stars, watchers, or forks.

9. Applications

- Digital Electronics coursework and laboratory demonstrations.
- Building blocks for larger combinational and sequential systems (e.g., priority encoders, multiplexer/demultiplexer control, memory address decoding).
- FPGA development and prototyping.
- VLSI front-end design and functional verification practice.
- Embedded systems and computer architecture (e.g., instruction/address decoding).

10. Conclusion

The project successfully demonstrates the design and verification of two fundamental combinational logic building blocks — a 4-to-2 encoder and a 2-to-4 decoder — using Verilog HDL. Both modules were implemented with concise, synthesizable always-block/case-statement constructs and validated against their complete truth tables using a single shared testbench. The simulation results matched the expected theoretical outputs for every input combination, confirming correct functional behavior.

As a compact, well-documented example of combinational circuit design and testbench-based verification, the project provides a solid, reusable reference for further study in digital design, FPGA implementation, and VLSI development.

11. Reference

M. Morris Mano & Michael D. Ciletti, *Digital Design*, 6th ed., Pearson, 2018.

Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed., Prentice Hall, 2003.

R. J. Tocci, N. S. Widmer, G. L. Moss, *Digital Systems: Principles and Applications*, 12th ed., Pearson, 2016.